

```
In [1]: import pandas as pd
import numpy as np
import openpyxl

import seaborn as sns
import matplotlib.pyplot as plt
import matplotlib.dates as mdates

from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_absolute_error, mean_absolute_percentage_error
from sklearn.preprocessing import StandardScaler, MinMaxScaler, OneHotEncoder
from statsmodels.tsa.stattools import grangercausalitytests

from sklearn.ensemble import RandomForestRegressor
from sklearn.pipeline import Pipeline
from sklearn.model_selection import GridSearchCV
from sklearn.feature_selection import RFE

plt.style.use('fivethirtyeight')

import datetime
from datetime import date, datetime, timedelta

import warnings
warnings.filterwarnings('ignore')
```

Функции

```
In [2]: # Функция для отображения графиков
def draw_result(y_test, pred, model):

    date_lst = [pd.to_datetime(dat).date() for dat in y_test.index]

    fig, ax = plt.subplots(figsize=(17,5))

    ax.plot(date_lst, y_test, color='green', linewidth=3)
    ax.plot(date_lst, pred, color='red', linewidth=3)

    plt.legend(["Фактическое значение", "Прогноз"], loc="upper right")
    plt.title(model)
    plt.gca().xaxis.set_major_locator(mdates.MonthLocator())

    ax.tick_params(axis='x', labelsize=9)
    plt.gcf().autofmt_xdate()
    plt.xticks(rotation = 0) # Поворачиваем надписи

# Функция для вывода ошибок RMSE и MAE
def rmse(fact, pred):
    diff = pred - fact
    different_squares = diff * diff
    mean_diff = different_squares.mean()
    rmse_val = np.sqrt(mean_diff)
    return round(rmse_val, 3)

def get_metrics(fact, pred):
    rmse_val = round(rmse(fact, pred), 3)
    mae_val = round(mean_absolute_error(fact, pred), 3)
    return f'RMSE: {rmse_val}, MAE: {mae_val}'

# Функция для создания лагов
def create_lag_features(data, lag_columns, lags):

    for col in lag_columns:
        for lag in lags:
            data[f'{col}_lag_{lag}'] = data[col].shift(lag)
    return data
```

Загрузка и свод статистики

```
In [3]: df = pd.read_excel('Свод.xlsx', index_col=0, header=0)[12:]
```

```
In [4]: round(df.describe().T, 2)
```

```
Out[4]:
```

	count	mean	std	min	25%	50%	75%	max
ИПЦ	180.0	100.59	0.73	99.46	100.31	100.48	100.75	107.61
ИПП_РФ	180.0	100.52	7.08	78.70	99.28	101.25	103.60	115.00
Обработка_РФ	180.0	101.02	10.52	66.00	99.48	102.30	105.82	119.00
Производство_электроэнергии_РФ	180.0	100.33	7.10	85.80	94.30	101.14	104.42	116.00
Торговля_НЕпроды_РФ	180.0	100.79	10.36	59.80	98.60	101.40	103.65	128.10
Индикатор_бизнес-климата_РФ	180.0	4.70	7.26	-18.61	-0.83	5.24	10.38	18.96
Ценовые_ожидания_РФ	180.0	16.34	7.07	6.60	11.52	14.38	20.42	49.43
Food Price Index	180.0	109.11	13.94	88.94	97.15	107.80	117.17	156.77
USD	180.0	57.82	20.77	27.91	32.84	62.43	72.72	103.47
Brent, price	180.0	78.35	24.07	26.63	59.60	77.00	103.29	124.54
Потребы_РФ	180.0	1.02	1.47	-5.00	0.28	1.13	1.97	4.04
Ипотека_РФ	180.0	1.64	0.99	-0.63	0.86	1.65	2.32	4.12
Средства_населения_РФ	180.0	1.08	1.72	-2.91	0.19	0.83	1.84	6.82

```
In [5]: y = df['ИПЦ']
X = df.drop(['ИПЦ'], axis=1)
```

```
In [6]: X.columns
```

```
Out[6]: Index(['ИПП_РФ', 'Обработка_РФ', 'Производство_электроэнергии_РФ',
              'Торговля_НЕпроды_РФ', 'Индикатор_бизнес-климата_РФ',
              'Ценовые_ожидания_РФ', 'Food Price Index', 'USD', 'Brent, price',
              'Потребы_РФ', 'Ипотека_РФ', 'Средства_населения_РФ'],
              dtype='object')
```

Формируем лаги

```
In [7]: lags = [x for x in range(12, 13)] # Лаги, которые хотим создать

lagged_df = create_lag_features(X, lag_columns=['ИПП_РФ', 'Обработка_РФ', 'Производство_электроэнергии_РФ',
        'Торговля_НЕпродукты_РФ', 'Индикатор_бизнес-климата_РФ',
        'Ценовые_ожидания_РФ', 'Food Price Index', 'USD', 'Brent, price',
        'Потребы_РФ', 'Ипотека_РФ', 'Средства_населения_РФ'], lags=lags)

lagged_df = lagged_df.drop(['ИПП_РФ', 'Обработка_РФ', 'Производство_электроэнергии_РФ',
        'Торговля_НЕпродукты_РФ', 'Индикатор_бизнес-климата_РФ',
        'Ценовые_ожидания_РФ', 'Food Price Index', 'USD', 'Brent, price',
        'Потребы_РФ', 'Ипотека_РФ', 'Средства_населения_РФ'], axis=1)[12:]
```

```
In [8]: round(lagged_df, 1)
```

Out[8]:

	ИПП_РФ_lag_12	Обработка_РФ_lag_12	Производство_электроэнергии_РФ_lag_12	Торговля_НЕпродукты_РФ_lag_12	Индикатор_бизнес-климата_РФ_lag_12	Ценовые_ожидания_РФ_lag_12	Food Price Index_lag_12	USD_
Дата								
2011-01-01	85.4	74.1	101.4	78.3	-6.4	16.7	101.9	
2011-02-01	100.0	108.8	90.2	97.2	9.2	13.0	99.3	
2011-03-01	112.6	118.3	101.6	108.9	15.8	10.5	96.6	
2011-04-01	94.2	96.1	87.1	100.6	11.2	9.5	97.1	
2011-05-01	99.3	100.3	93.4	102.1	12.6	9.3	96.5	
...	...	...	...	...	...	...	...	...
2024-08-01	102.2	103.1	102.9	105.1	9.7	26.4	117.1	
2024-09-01	100.7	101.5	97.0	95.8	4.9	26.4	117.0	
2024-10-01	105.0	102.9	113.8	101.6	3.3	26.0	116.1	
2024-11-01	101.1	101.8	105.0	95.5	-0.3	28.5	116.0	
2024-12-01	110.0	111.3	111.7	114.1	3.1	28.9	114.5	

168 rows × 12 columns



Random Forest без подбора гиперпараметров

```
In [9]: results = pd.DataFrame()

n = 12

y_train = y[12:-(n)]
X_train = lagged_df[:-n]
y_test = y[-n:]
X_test = lagged_df[-n:]
```

```
In [10]: X_train
```

Out[10]:

	ИПП_РФ_lag_12	Обработка_РФ_lag_12	Производство_электроэнергии_РФ_lag_12	Торговля_НЕпродукты_РФ_lag_12	Индикатор_бизнес-климата_РФ_lag_12	Ценовые_ожидания_РФ_lag_12	Food Price Index_lag_12	USD_
Дата								
2011-01-01	85.4	74.1	101.400000	78.3	-6.429515	16.698669	101.856476	29.8
2011-02-01	100.0	108.8	90.200000	97.2	9.197589	12.967103	99.310448	30.1
2011-03-01	112.6	118.3	101.600000	108.9	15.772815	10.546825	96.559061	29.5
2011-04-01	94.2	96.1	87.121336	100.6	11.177420	9.496956	97.112079	29.1
2011-05-01	99.3	100.3	93.367901	102.1	12.601781	9.307109	96.453063	30.4
...	...	...	...	...	...	...	...	...
2023-08-01	101.6	102.0	103.506000	105.5	7.889594	15.071063	135.698198	60.3
2023-09-01	100.6	101.0	99.514000	95.7	-2.442278	16.191439	134.367615	59.8
2023-10-01	105.4	104.6	110.524000	100.7	-3.043735	19.484281	133.716157	61.1
2023-11-01	102.1	102.9	106.408000	100.7	-5.230274	24.196125	133.043806	60.8
2023-12-01	111.6	114.9	109.782000	114.5	-0.954207	23.237067	130.226296	65.8

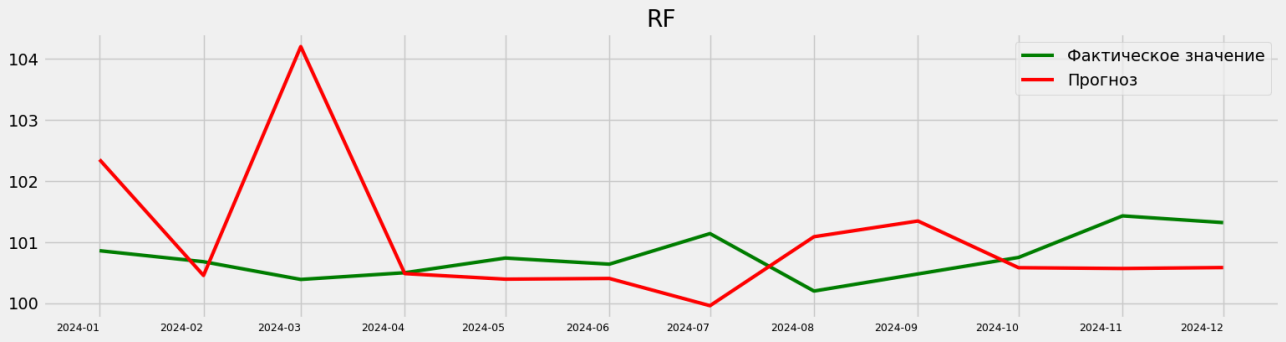
156 rows × 12 columns



```
In [11]: rf = RandomForestRegressor()
rf.fit(X_train, y_train)
pred = rf.predict(X_test)
results_1 = pred
```

```
In [12]: print(get_metrics(y_test, results_1))
draw_result(y_test, results_1, 'RF')
```

RMSE: 1.329, MAE: 0.901



Random Forest

```
In [13]: %%time
n = 12
y_train = y[12:-n]
X_train = lagged_df[:-n]
y_test = y[-n:]
X_test = lagged_df[-n:]

pipeline_rf = Pipeline([
    ('scaler', StandardScaler()),
    ('feature_selection', RFE(estimator=RandomForestRegressor(random_state=42))), #, n_features_to_select=4
    ('classifier', RandomForestRegressor(bootstrap=True))
])

param_grid_rf = {
    'classifier__n_estimators': [100, 300, 500], # Количество деревьев
    'classifier__max_depth': [3, 5, 7, 10], # Максимальная глубина дерева
    'classifier__min_samples_split': [2, 3, 5, 8], # Минимальное количество выборок для разбиения узла
    'classifier__min_samples_leaf': [2, 3, 5, 8], # Минимальное количество выборок в листовом узле
    'classifier__max_features': [2, 4, 6, 7, 8, 10],
}

grid_search_rf = GridSearchCV(
    estimator=pipeline_rf,
    param_grid=param_grid_rf,
    cv=2, # Кросс-валидация с 5 разбиениями
    scoring='neg_mean_absolute_error', # Метрика: MAE
    verbose=2, # Отображение прогресса
    n_jobs=-1 # Использование всех доступных процессоров
)

grid_search_rf.fit(X_train, y_train)

pred = grid_search_rf.predict(X_test)

results_2 = pred

best_pipeline_rf = grid_search_rf.best_estimator_
rfe_step_rf = best_pipeline_rf.named_steps['feature_selection']
selected_features_mask_rf = rfe_step_rf.support_
selected_features_rf = X_train.columns[selected_features_mask_rf]

print("Отобранные признаки RFE:", selected_features_rf)
print()
print("Лучшие параметры:", grid_search_rf.best_params_)
print()
print("Лучший MAE (на обучающей выборке):", round(-grid_search_rf.best_score_, 2))
print()
# print(f'RMSE на тесте: {0.508}')
print(f'MAE на тесте: {round(mean_absolute_error(y_test, results_2), 2)}')
print()
print("Прогноз:", results_2)
print()

Fitting 2 folds for each of 1152 candidates, totalling 2304 fits
Отобранные признаки RFE: Index(['ИПП_РФ_lag_12', 'Обработка_РФ_lag_12',
    'Индикатор_бизнес-климата_РФ_lag_12', 'USD_lag_12', 'Ипотека_РФ_lag_12',
    'Средства_населения_РФ_lag_12'],
    dtype='object')

Лучшие параметры: {'classifier__max_depth': 5, 'classifier__max_features': 8, 'classifier__min_samples_leaf': 5, 'classifier__min_samples_split': 3, 'classifier__n_estimators': 300}

Лучший MAE (на обучающей выборке): 0.37

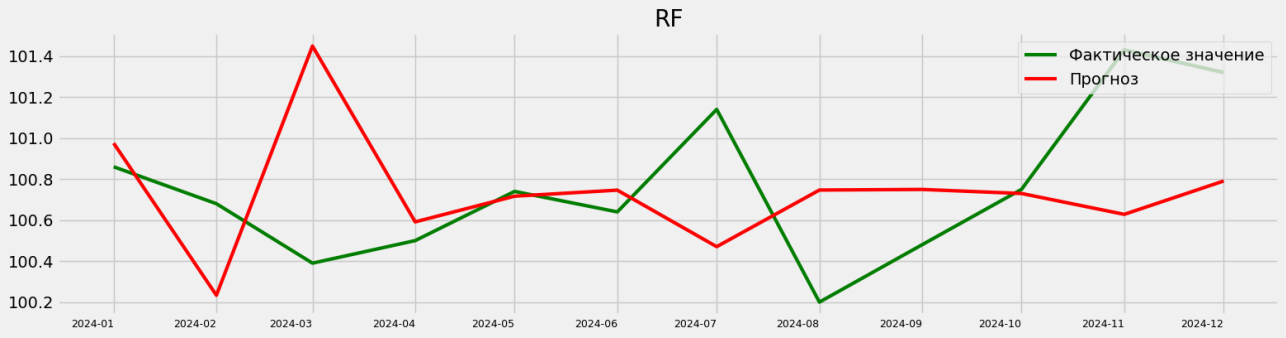
MAE на тесте: 0.39

Прогноз: [100.9750639 100.23337126 101.44884364 100.59133834 100.71663841
100.74612172 100.47042099 100.74659068 100.74953432 100.72977993
100.6280492 100.7906884 ]

CPU times: total: 11.2 s
Wall time: 7min

In [14]: print(get_metrics(y_test, results_2))
draw_result(y_test, results_2, 'RF')

RMSE: 0.508, MAE: 0.39
```



In []: